MMA3001 – Numerical Methods and Machine Learning

Overview

The individual project is a major assessment component worth **25% of your mark in this unit**. It is designed to assess a students' ability to apply numerical methods, machine learning, computer vision, data analysis, optimisation, and any combination of these techniques, to meaningful engineering problems.

The emphasis is **not** on selecting the most complicated project or challenging solution. Students will be assessed primarily on:

- demonstrated **individual learning** (EA competency - lifelong learning)
- the **quality** and **organisation** of their computational work;
- the **validity** of their evaluation;
- the **justification** for their technical decisions;
- the **lessons** drawN from their results;
- their ability to **communicate** their work clearly and professionally.

Students may complete either:

- One of the suggested projects (details to be announced), or
- A self-selected project that aligns with the unit learning outcomes.

Self-selected projects must be discussed with the Unit Coordinator prior to commencement to ensure suitability, scope, and feasibility.

A project **does not** need to produce a perfect solution to earn a strong result. An unsuccessful or partially successful method can still demonstrate excellent engineering work if the student validates it honestly, identifies its limitations, explains what was learned and proposes well-justified improvements.

Document Links:

Overview
Project Selection
    Possible Project Areas
Project Requirements
Assessment Deliverables
Support and Guidance

Project Selection

Students will select their project by the end of **Week 5**, following the mid-semester assessment.

Students may either:

1. select from a set of projects and datasets provided by the teaching team; or
2. propose their own engineering problem and dataset.

Note: All projects are Open-End as required by accreditation to meet Life-long learning and Open-Lab requirements.

Student-proposed projects and datasets must be approved by the teaching team before substantial work begins. Approval will consider:

- suitability for the unit;
- clearly defined inputs and outputs;
- achievable scope;
- availability of suitable validation evidence;
- data ownership, licensing and privacy;
- whether credible alternative solutions can be compared.

The teaching team may ask a student to reduce or modify the scope of an overly complex proposal.

Each project must include:

- A clearly defined engineering problem relevant to mechanical or aerospace engineering.
- Application of at least one numerical method and/or machine-learning technique.
- Evidence of:
    - Model development
    - Validation and verification
    - Sensitivity analysis or optimisation (where appropriate)
    - Interpretation of results in an engineering context
- A reproducible workflow using version control (GitHub required). ● Appropriate use of computational tools (Python, Jupyter, numerical libraries, ML frameworks).
- Acknowledgement and details of AI usage.

There are no formal project milestones. However, expected progress and project guidance will be discussed during the remaining teaching weeks.

Possible Project Areas

Possible provided projects may include:

Apple-tree reconstruction

- **Input:** RGB-D observations of apple trees collected over three seasons.
- **Output:** Estimated three-dimensional tree skeletons.
- **Validation:** Comparison with ground-truth skeletons that may be partially obscured by leaves.

Frozen-meat packaging quality

- **Input:** RGB images of frozen-meat products and packaging.
- **Output:** A report describing meat quality and packaging quality.
- **Validation:** Comparison with labelled quality assessments or known inspection outcomes.

If you prefer, you are welcome to work on simulations rather than data analysis - please discuss this when proposing your own project.

Details of sample topics will be provided in separate documents.

Project Requirements

Every project must consist of a function GitHub, which contains your code, documentation and readme files.

The Documentation will act as a project report and should include the

following. **1. Engineering problem**

Your Project documentation must explain:

- the engineering context;
- the problem being addressed;
- why the problem matters;
- the intended use of the computational solution;
- the limitations of the selected scope.

2. Inputs and outputs

Documentation - must clearly define:

- the input data accepted by the solution;
- the expected format, units and domain of the inputs;
- the outputs produced;
- the meaning and intended use of each output;
- how invalid, missing or unsupported inputs are handled.

The documentation should address top-level inputs and outputs of the project. Later part of the documentation should address specific inputs and output of functions you have developed. You can use Docstrings and Documentation generators to assist with this.

A computational method cannot be evaluated properly unless its inputs and outputs are unambiguous.

3. Computational solution

Students must develop a functioning and reproducible computational workflow.

The selected approach may involve any defensible computational method. Students must explain:

- how the method works at an appropriate level;
- why it was selected;
- its important assumptions;
- its settings or parameters;
- how it was implemented;
- its known limitations.

4. Alternative solutions

Students must identify and compare credible alternative approaches.

At least one alternative or baseline solution should be considered. The comparison may address:

- accuracy;
- robustness;
- computational time;
- memory use;
- implementation complexity;
- interpretability;
- scalability;
- suitability for the intended engineering application.

The purpose is not to test every possible method. It is to demonstrate that the final approach was selected using evidence rather than preference alone.

5. Validation

Every project must include a clear and justified validation method.

Depending on the project, validation may use:

- known ground truth;
- withheld test data;
- independent experimental data;
- repeated measurements;
- comparison with a trusted benchmark;
- comparison with an established computational method;
- cross-validation;
- physical constraints or conservation relationships;
- sensitivity and robustness testing;
- manual inspection supported by defined criteria.

Students must explain:

- what was validated;
- why the validation method is appropriate;
- which metrics or acceptance criteria were used;
- whether the validation information influenced model selection;
- what the validation cannot establish.

A numerical result or visually convincing output is not sufficient evidence of accuracy. 5

6. Optimisation and performance

Students must analyse and, where appropriate, improve the performance of their solution.

Relevant considerations may include:

- solution accuracy;
- computational runtime;
- memory requirements;
- robustness;
- numerical stability;
- model or algorithm settings;
- input resolution;
- data requirements;
- repeatability;
- scalability;
- trade-offs between performance measures.

Optimisation should be relevant to the project. Students are not expected to optimise every possible characteristic.

Students should make use of profilers, and timing routines, and where appropriate estimates of arithmetic intensity and required number of FLOPS associated with a method to help explain and justify the performance of their code.

The report must compare the original or baseline solution with the improved or selected solution and justify the final design.

7. AI use and reflection

Students may use AI tools throughout the project.

All material AI use must be disclosed. The report must include an AI reflection that explains:

- which AI tools were used;
- what they were used for;
- the approximate level of AI contribution;
- why AI was used for those tasks;
- how AI-generated material was checked;
- which important decisions remained the student's responsibility;
- any errors, limitations or unhelpful suggestions produced by AI;
- how AI use affected the student's understanding or workflow.

Students remain responsible for all submitted code, results, claims, references and engineering decisions. Disclosure does not replace verification.

There is **no penalty** for use of **AI** while **undertaking the project,** as long as it is done responsibly**.** You will **not** be able to use **AI** to answer questions during the **presentation and interview.**

Each project must include:

- A clearly defined engineering problem relevant to mechanical or aerospace engineering.
- Application of at least one numerical method and/or machine-learning technique taught in the unit.
- Evidence of:
    - Model development
    - Validation and verification
    - Sensitivity analysis or optimisation (where appropriate)
    - Interpretation of results in an engineering context
- A reproducible workflow using version control (GitHub required). ● Appropriate use of computational tools (Python, Jupyter, numerical libraries, ML frameworks).

Assessment Deliverables

(see Unit Overview Spreadsheet for due dates)

Written Project Documentation

Submit an individual written document of **5-10 pages maximum**.

The report should communicate:

1. the engineering problem;
2. the inputs, outputs and data;
3. the computational approach;
4. the validation method and results;
5. the comparison and optimisation work;
6. the limitations and lessons learned;

7. the AI-use reflection;
8. References to other data sets, methods, reports, as appropriate.

Note: References and Appendices will not count towards the 10-page

limit. This must be **submitted** via Moodle.

This documentation forms the basis of your presentation and you will be expected to answer questions based on this material. Your **mark** will be based on your **presentation** and ability to **respond to questions** about this work.

GitHub repository

You are required to maintain a GitHub repository during development of your project for version control and for sharing and distributing your code, as aligned with industry practice.

Your repository should include:

1. ReadMe file
2. HTML documentation of your source code (can be generated by pdoc or similar)
3. Licensing agreement
4. Source code - functional main branch.
    a. Code should include inline comments and Docstring.
5. Test functions and Test report (can be generated by pytest)

Submit an accessible link to the final repository. The repository must contain the code and documentation needed to review the computational work.

A zipped **CLONE** of this repository must be **submitted** via Moodle

Presentation/Demonstration

*“How did you verify that your numerical method was implemented correctly (e.*Deliver an individual **5 minute presentation** of your project + **3 mins questions** in Week 14 during the examination period.

A series of appointment times will be released later in the semester. You will be required to book and attend one of these times. **Marks** will not be released until this is completed.

The presentation should include:

- a brief explanation of the engineering problem - primary input and output; ● include a demonstration of your GitHub and structure, documentation of your code; ● evidence of code validation and testing;
- You chosen computation approach - reason you chose this;
- the principal limitations or lesson learned;

- the student's use and evaluation of AI.

You will then be asked one or more questions depending on the nature of your problem. For example:

“What physical or engineering system did your numerical model represent, and what assumptions were required to make the problem tractable?”

“Which parameters or boundary conditions had the greatest influence on the behaviour of your solution, and how did you verify they were realistic”

“How did you ensure numerical stability in your implementation?”

“Describe any convergence criteria you used. Why are they appropriate for this problem?”

“What are the main sources of numerical error in your solution, and how did you quantify or control them?”

g., comparison with analytical solutions, manufactured solutions, benchmark problems)?”

“Which part of your solution is most vulnerable to numerical failure, and how would you mitigate that risk?”

Support and Guidance

Students are encouraged to:

- Seek feedback during practical classes.
- Discuss project ideas early
- Use recommended tools (GitHub, Jupyter, testing frameworks)
- Engage in self-directed learning consistent with professional engineering practice
- Post question to edStem.

Assessment Item
Excellent Good Satisfactory Poor

Engineering problem, inputs and outputs
A clear, professional, and complete
description of the engineering problem and all primary
inputs/outputs.
Engineering context is well-explained; the problem is precisely defined; its
importance and

intended use are explicit; scope and limitations are clearly justified. Inputs and outputs are fully specified, including format, units,
domains, meaning, and intended use.
A clear description of the engineering
problem and most inputs/outputs.
Engineering context and motivation are explained with minor gaps. Inputs and outputs are defined with reasonable clarity, including format and units, though some details (domains, limitations) may be incomplete.
A basic description of the engineering
problem but lacks depth or clarity.
Motivation or intended use may be vague. Inputs and outputs are partially defined but missing important details such as units, domains, or meaning.
Engineering problem description is unclear or missing. Inputs and
outputs not defined or incorrect. No explanation of context, motivation, intended use, or
limitations. No discussion of input formats, units, or domains..

Assessment Item
Excellent Good Satisfactory Poor

**GitHub repository structure,
commits and
ReadME**
Professional, clear, and fully reproducible repository structure. README provides complete orientation and workflow
instructions.

Evidence: logical folder hierarchy;

consistent naming; clear README with purpose, usage, environment, dependencies; reproducible workflow; appropriate handling of data; evidence of branching and/or insightful commits. Well-organised repository with a mostly complete README.

Evidence: clear structure; meaningful commits; README covers main points; minor gaps in workflow or dependency clarity. Repository structure present but inconsistent. README incomplete or unclear.

Evidence: partial structure; README missing key details; workflow reproducible only with assistance. inconsistent structure; sporadic commits; readability issues. Disorganised or not reproducible, missing or unusable README.

Evidence: confusing structure; few commits; missing data/dependencies; workflow cannot be reproduced;

Assessment Item

Excellent Good Satisfactory Poor

Inline comments, docstrings, and HTML documentation

Code is highly readable and professionally documented. Inline comments clarify intent rather than restating code. Docstrings follow a consistent style (Google, NumPy, or Sphinx) and fully describe parameters, return values, exceptions, and usage.

HTML documentation (e.g., Sphinx, MkDocs, pdoc) is complete, well-structured, documentation and code remain in sync.

Code is readable with generally helpful comments. Docstrings present for most functions/classes, though detail or consistency may vary. HTML documentation exists and covers main modules but may have minor gaps or outdated sections.

Documentation is present but inconsistent or incomplete. Inline comments are sporadic or overly literal. Docstrings missing for many functions or lacking detail. HTML documentation minimal, partially generated, or missing key modules.

Little to no meaningful documentation. Inline comments absent or misleading. Docstrings missing or incorrect or non-standard. No HTML documentation or broken/incomplete output. Code is difficult to understand without external explanation.

Assessment Item
Excellent Good Satisfactory Poor

Validation and engineering credibility
Rigorous validation demonstrating strong engineering judgement.

Evidence: clear inputs/outputs; appropriate validation design; correct metrics; accurate results; comparison with ground truth/withheld data/benchmarks/phy sical constraints; identification of unsupported inputs and failure modes; honest limitations.
Appropriate and mostly complete validation.

Evidence: defined inputs/outputs; reasonable validation approach; mostly appropriate metrics; clear results; some comparison - some element left unexplored; limitations acknowledged.
Superficial or incomplete validation.

Evidence: partially defined inputs/outputs; weak metrics; limited comparison; limited discussion of

limitations.
Missing or incorrect
validation.

Evidence: no clear
validation; inappropriate metrics; misleading
results; no comparison; no limitations.

Assessment Item
Excellent Good Satisfactory Poor

Justification of Computational Approach
Deep technical
reasoning with
credible baselines and evidence-based optimisation.

Evidence: clear
baseline; justified method selection; quantitative analysis (accuracy, runtime, memory, robustness, scalability);
evidence-supported improvements;
thoughtful trade-offs; conclusions
grounded in results.
Solid reasoning with some optimisation or comparison.

Evidence: baseline present; mostly

justified method selection; some quantitative analysis; improvements attempted; trade-offs discussed.
Limited comparison or optimisation.

Evidence: unclear baseline; weak justification; minimal quantitative analysis; speculative improvements; brief trade-offs.
No meaningful comparison or justification.

Evidence: no baseline; unjustified method selection; no performance analysis; no trade-offs; unsupported conclusions.

Assessment Item
Excellent Good Satisfactory Poor

AI use and critical reflection
Fully disclosed, critically evaluated, and clearly subordinate to the student's own work. The student was clearly driving the AI process to arrive at the solution.

Evidence: complete disclosure; justification; verification of AI

content; identification of errors; clear evidence of understanding; thoughtful reflection on AI's impact.

Disclosed and mostly well-managed, clear understanding of AIs output, the student has strong overall responsibility for the output.

Evidence: disclosure; basic justification; some verification; adequate reflection; understanding evident.

Disclosed but weakly justified or poorly verified.

Evidence: minimal disclosure; limited verification; superficial reflection; uncertainty about ownership.

Undisclosed or inappropriate AI use.

Evidence: no disclosure; reliance without verification; unrecognised errors; unclear understanding; no reflection.